Security in system design should not be treated as a separate component such as "add OAuth at the API Gateway." It is a **cross-cutting architectural concern** that must be designed into every layer: identity, network, application, data, infrastructure, observability, and operations.

For a MAANG+/Staff-level system-design interview, I would frame it as:

> **Authenticate → Authorize → Validate → Protect → Detect → Respond → Recover**

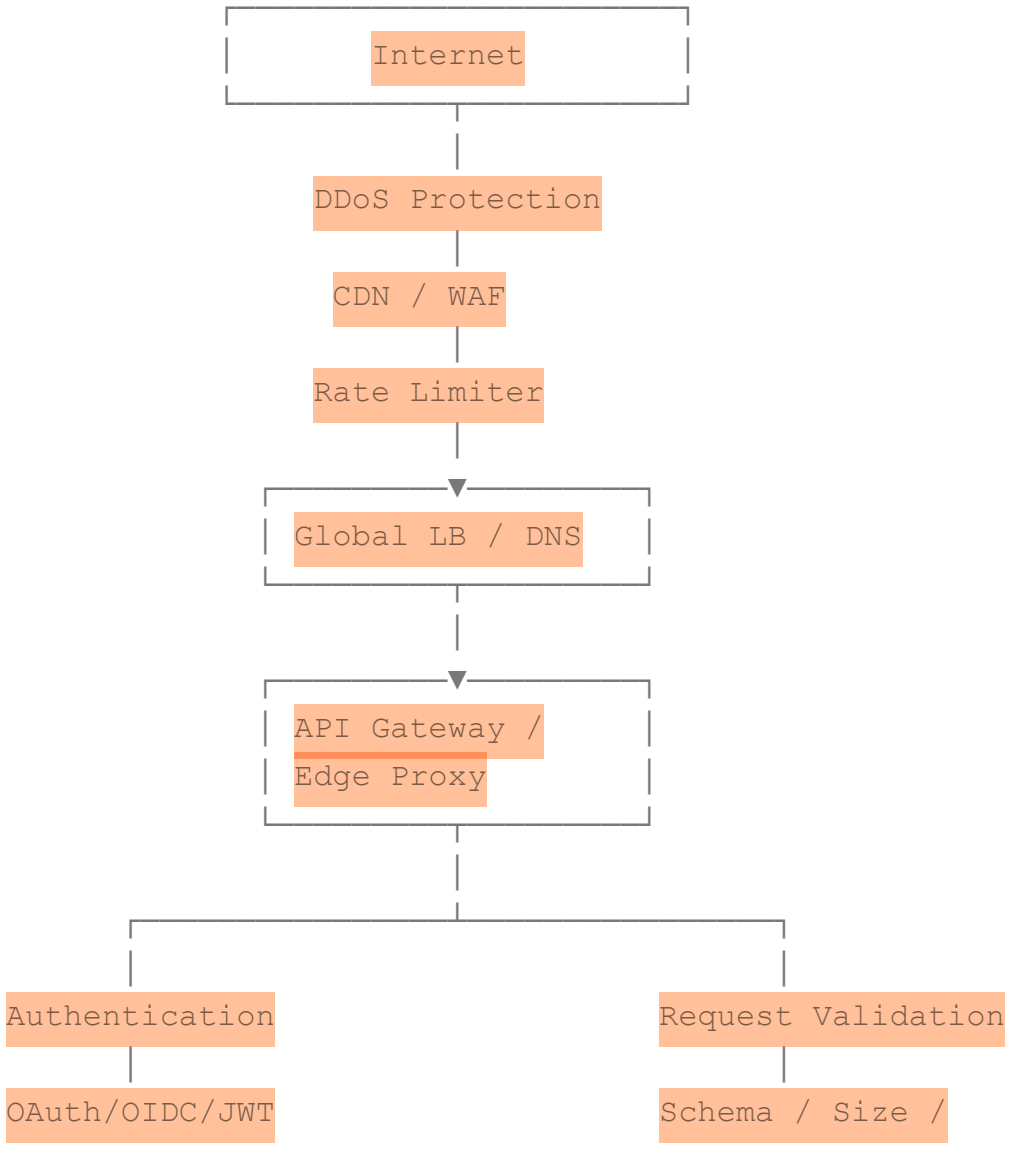
And, at scale:

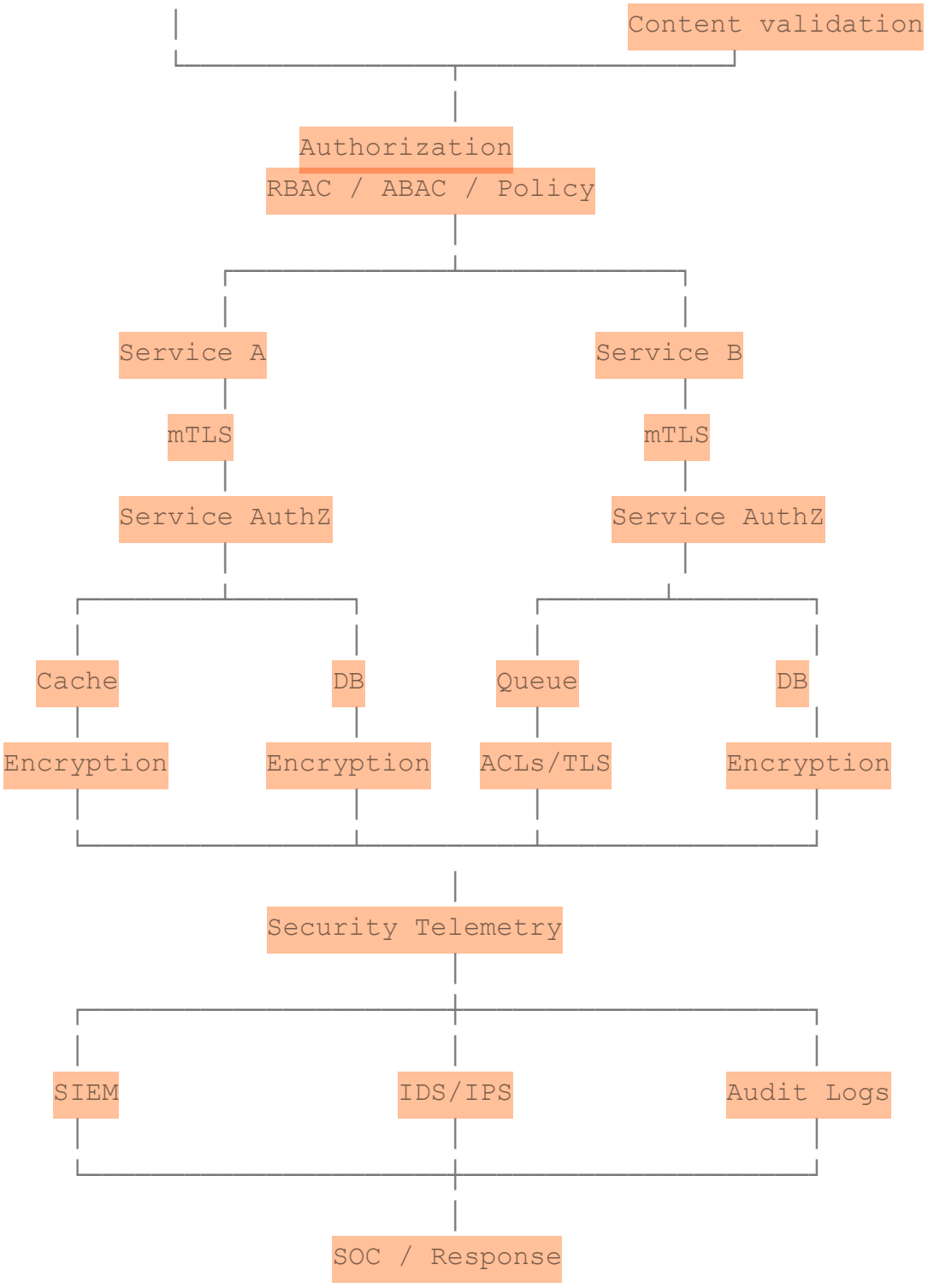
> **Centralize security policy and identity, but distribute enforcement close to the resource being protected.**

---

1. The big picture

A large-scale secure system can look like this:





But this is only the beginning. The important part is understanding **where each security responsibility belongs and how it scales**.

---

2. Start with a threat model

Before selecting technologies, identify:

Assets

What are we protecting?

- User identities
- Access tokens
- PII
- Financial information
- Business data
- Source code
- Secrets
- Encryption keys
- Infrastructure
- Databases
- Messages
- Internal APIs
- Audit records

Threat actors

- Unauthenticated attacker
- Compromised user
- Compromised service
- Malicious insider
- Stolen credentials
- Botnet
- Supply-chain attacker
- Compromised dependency
- Ransomware
- Cloud account compromise

Attack surfaces

- Internet APIs
- Web application
- Mobile application
- Admin APIs
- Internal APIs
- WebSockets
- MQTT
- Kafka
- Databases

```
Object storage
CI/CD
Cloud control plane
Third-party integrations
Developer machines
```

Then identify threats using something such as:

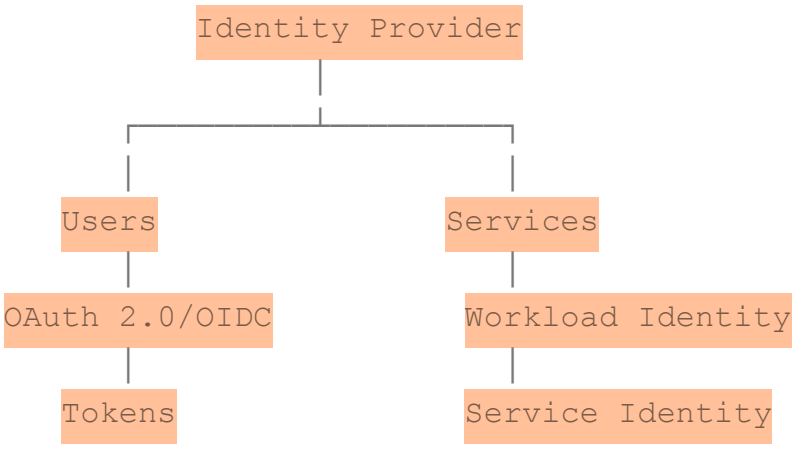
```
Spoofing
Tampering
Repudiation
Information disclosure
Denial of service
Elevation of privilege
```

This is essentially the **STRIDE** mindset.

---

3. Identity is the foundation

A scalable security architecture normally starts with a centralized identity system.



For humans:

```
User
 ↓
Identity Provider
 ↓
Authentication
 ↓
```

```
Access Token
  ↓
API
```

For services:

```
Service A
    ↓
Workload Identity
    ↓
Service credential / certificate
    ↓
Service B
```

---

4. Authentication vs authorization

This distinction is extremely important in system-design interviews.

Authentication

> **Who are you?**

Examples:

```
Password
MFA
Passkeys
OAuth 2.0
OIDC
Client certificates
Workload identity
```

Authorization

> **What are you allowed to do?**

Examples:

```
RBAC
ABAC
ACL
Policy-based authorization
Resource ownership
Tenant isolation
```

Never confuse:

```
JWT validation ≠ authorization
```

A valid JWT only establishes that the token is valid and represents some identity/claims.

You still need:

```
Can user U perform operation X
on resource R
in tenant T?
```

---

5. Centralize identity, decentralize enforcement

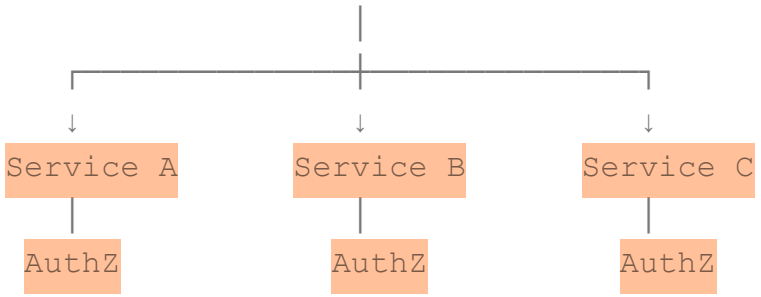
This is one of the most important architectural principles.

Avoid:

```
Every microservice
      ↓
Its own user database
      ↓
Its own authentication
```

Instead:

```
                Identity Provider
                       |
                  Access Token
```



The identity provider is centralized.

But services independently enforce authorization.

Why?

Because if every authorization decision requires calling a central service:

```
Request
  ↓
Service A
  ↓
Authorization Service
  ↓
Service A
  ↓
Database
```

the authorization service becomes:

- latency bottleneck
- availability dependency
- scaling bottleneck
- potential single point of failure

Instead, use:

```
Central policy
      ↓
Distributed enforcement
```

---

6. API Gateway security

The edge is your first major security boundary.

Typical flow:

```
Internet
    ↓
DDoS Protection
    ↓
CDN
    ↓
WAF
    ↓
Load Balancer
    ↓
API Gateway
    ↓
Microservices
```

The API Gateway can perform:

```
TLS termination
Authentication
JWT validation
Rate limiting
Request size limits
Schema validation
IP filtering
Bot detection
Header normalization
API key validation
Tenant identification
Routing
Audit metadata
```

But don't put **all** authorization there.

For example:

```
Gateway:
    Is this token valid?

Service:
    Can this user modify this specific order?
```

---

7. WAF

A Web Application Firewall protects against common application-layer attacks.

Examples:

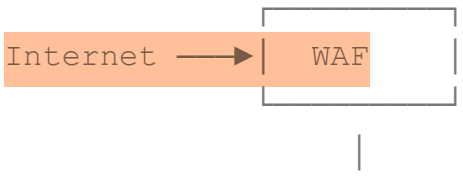
```
SQL injection
XSS
Malicious payloads
Protocol anomalies
Known attack signatures
Bot traffic
Request flooding
```

Architecture:

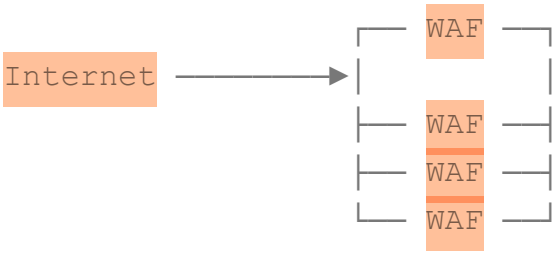
```
Client
  ↓
DDoS protection
  ↓
WAF
  ↓
CDN
  ↓
LB
  ↓
Gateway
```

The WAF should be horizontally scalable.

Do not create:



Instead:



or use a cloud-managed distributed edge service.

---

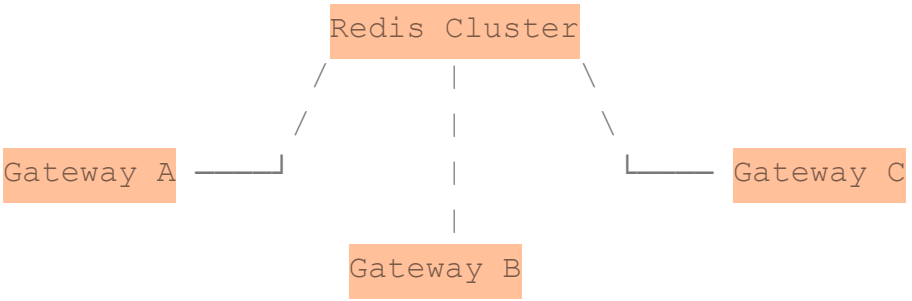
8. Rate limiting

Rate limiting is both a scalability mechanism and a security mechanism.

It protects against:

```
Brute-force attacks
Credential stuffing
API abuse
Scraping
DoS
Resource exhaustion
```

A basic architecture:



Common algorithms:

Fixed window

```
100 requests / minute
```

Simple but suffers from boundary bursts.

Sliding window

More accurate but more expensive.

Token bucket

Excellent for APIs:

```
capacity = 1000
refill = 100 tokens/sec
```

Allows controlled bursts.

Leaky bucket

Useful when you want smoother traffic.

At large scale, you may use:

```
Global rate limit
+
Regional rate limit
+
Per-user limit
+
Per-IP limit
+
Per-tenant limit
+
Per-endpoint limit
```

For example:

```
IP:
    1000 req/min

User:
    500 req/min

Tenant:
    10000 req/min
```

```
POST /login:
    10 req/min/IP

POST /payment:
    20 req/min/user
```

---

9. Distributed rate limiting

A common interview problem is:

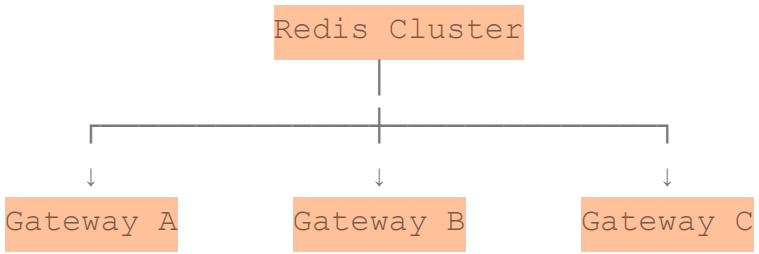
> How do you rate-limit when you have 1,000 API Gateway instances?

This won't work correctly:

```
Gateway 1 → local counter
Gateway 2 → local counter
Gateway 3 → local counter
```

because each instance sees only part of the traffic.

You can use:



with atomic operations/Lua scripts.

For extreme scale, hierarchical limiting is better:

```
Global quota
    ↓
Region quota
    ↓
Gateway quota
    ↓
Local token bucket
```

This reduces centralized coordination.

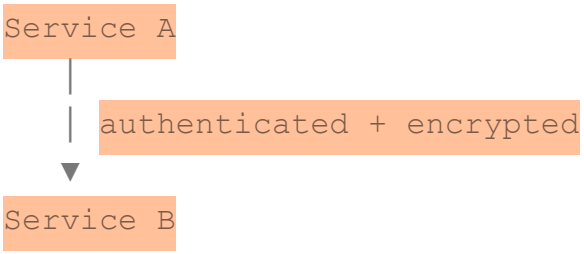
---

10. Service-to-service security

Don't assume:

> "It's inside the VPC, therefore it's trusted."

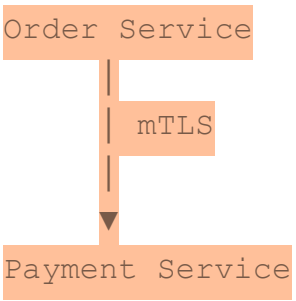
Modern architecture should use a **zero-trust** model.



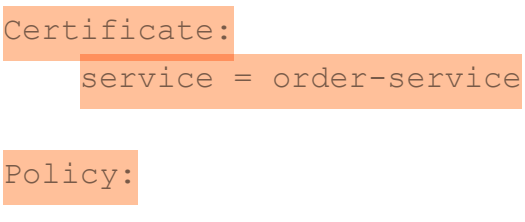
Use:



For example:



Payment Service verifies:



```
order-service may invoke
POST /payments
```

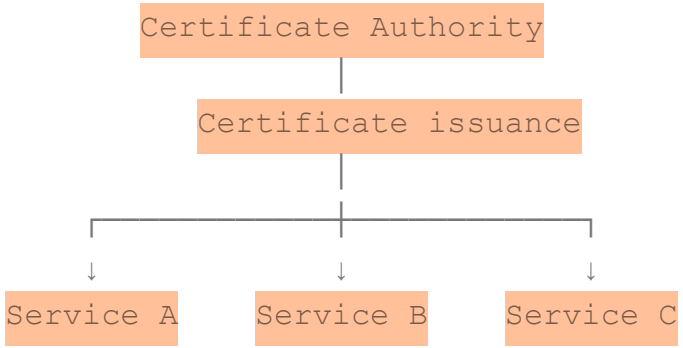
---

11. mTLS at scale

Manually managing certificates is painful.

Use automated certificate issuance and rotation.

Conceptually:



Certificates should have relatively short lifetimes.

Rotation becomes:

```
old certificate
      ↓
new certificate
      ↓
overlap
      ↓
old expires
```

No manual intervention.

A service mesh can help automate:

```
mTLS
identity
certificate rotation
traffic policy
```

```
authorization
telemetry
```

But a service mesh itself isn't a security architecture. It is an enforcement mechanism.

---

12. Authorization models

RBAC

Role-based:

```
Admin
Manager
Employee
Customer
```

Example:

```
Admin → DELETE users
Manager → UPDATE users
Employee → READ users
```

Simple and effective.

ABAC

Attribute-based:

```
user.department == resource.department
AND
user.role == "MANAGER"
AND
request.action == "READ"
```

Much more expressive.

ReBAC

Relationship-based:

```
Alice
  │
  └── owner_of ──▶ Document 123
```

Then:

```
Alice can edit Document 123
```

This becomes useful for:

```
Google Drive-like systems
GitHub-like systems
Collaboration platforms
Multi-tenant SaaS
```

---

13. Policy engine

At enterprise scale, authorization rules can become complex.

Instead of embedding everything:

```
if (user.isAdmin() ||
    user.department().equals(resource.department()) ||
    ...)
```

you can externalize policy:

```
Application
     │
     │ authorization request
     ▼
Policy Engine
     │
```

```
        ▼
  Allow / Deny
```

Conceptually:

```
  Subject
  Resource
  Action
  Context
        |
        ▼
  Policy
        |
        ▼
  Decision
```

For example:

```
  subject = user-123
  resource = invoice-456
  action = READ
  tenant = tenant-7
  location = IN
```

Policy:

```
  ALLOW if
  subject.tenant == resource.tenant
  AND
  subject.role IN ["ADMIN", "ACCOUNTANT"]
```

---

14. Don't make authorization synchronous unnecessarily

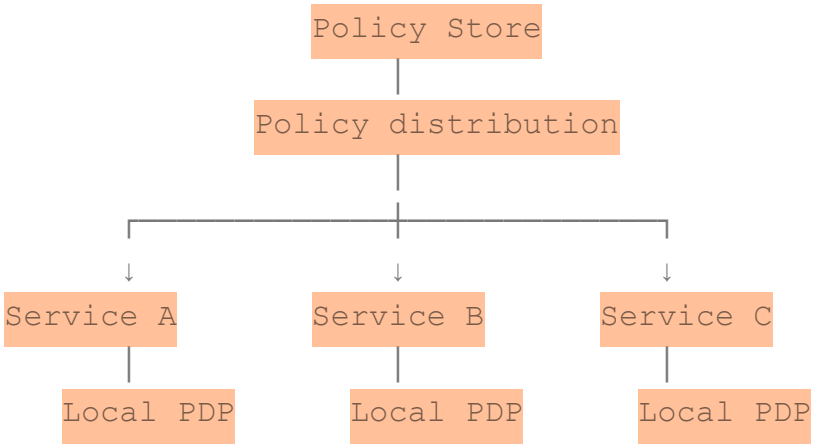
This is a critical scaling decision.

Bad:

```
API
 ↓
Auth Service
 ↓
Policy DB
 ↓
API
 ↓
Business Service
```

Every request depends on the policy service.

Better:



Policy changes propagate asynchronously.

For extremely sensitive operations, synchronous centralized checks may still be justified.

---

15. JWT design

JWTs are useful, but large-scale systems often misuse them.

A JWT contains:

```
Header
Payload
Signature
```

For example:

```
{
  "sub": "user-123",
  "tenant": "tenant-1",
  "roles": ["USER"],
  "exp": 1780000000
}
```

Advantages:

```
Stateless
Easy horizontal scaling
No session lookup for every request
```

But JWTs have an important property:

> Once issued, changing server-side permissions doesn't automatically invalidate an already-issued token.

Therefore:

```
short-lived access token
+
refresh token
+
token revocation strategy
```

is often preferable.

---

16. Session-based authentication

JWT isn't always better.

For traditional browser applications:

```
Browser
   ↓
Session Cookie
   ↓
Session Store
```

The session store can be:

```
Redis Cluster
```

Then:

```
User
 ↓
LB
 ↓
Application A
 ↓
Redis
```

or:

```
User
 ↓
LB
 ↓
Application B
 ↓
Redis
```

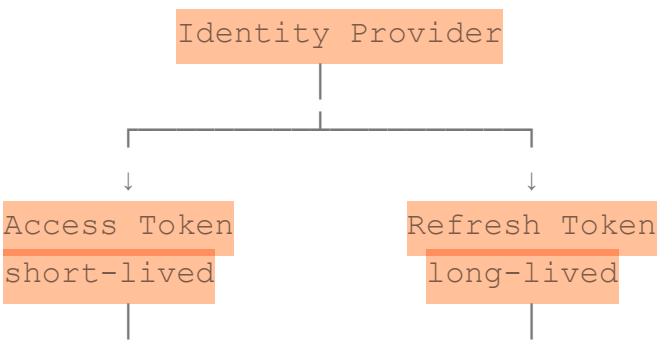
No sticky sessions required.

This is often simpler when server-side session invalidation is important.

---

17. Access tokens vs refresh tokens

A common architecture:





Example:

```
Access token:
5–15 minutes

Refresh token:
days/weeks
```

The exact lifetime depends on risk and product requirements.

---

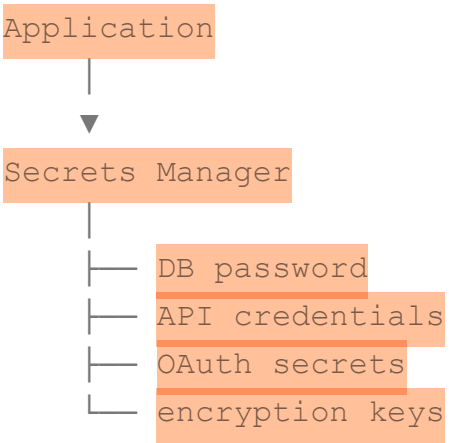
18. Secrets management

Never do this:

```
database:
  password: "my-password"
```

in source control.

Instead:



Examples of secrets:

```
Database passwords
API keys
Private keys
Certificates
OAuth client secrets
Cloud credentials
Third-party tokens
```

Secrets should have:

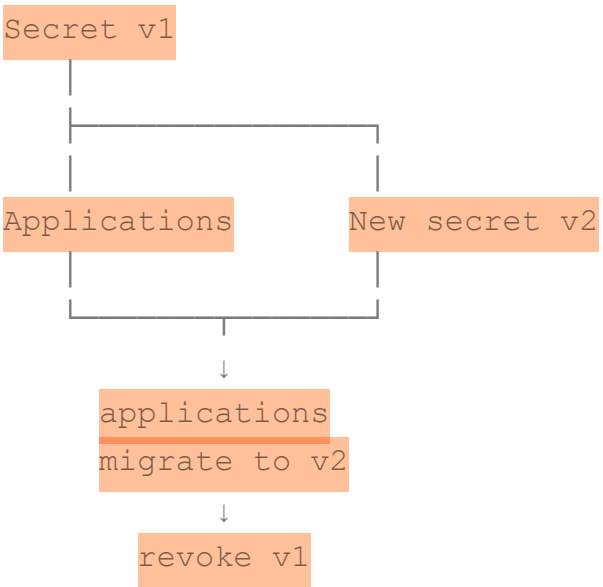
```
Encryption
Access control
Rotation
Auditing
Versioning
Expiration
```

---

19. Secret rotation

Don't design rotation as a maintenance activity.

Design it as a normal runtime operation:



Applications should support overlapping credentials where possible.

---

20. Encryption

You generally need:

Encryption in transit

```
TLS
mTLS
```

Encryption at rest

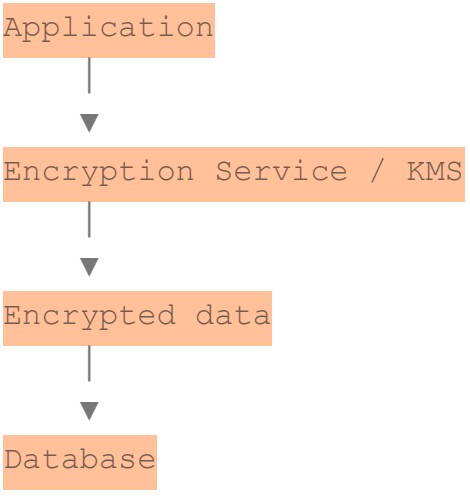
```
Database encryption
Disk encryption
Object storage encryption
Backup encryption
```

Application-level encryption

For particularly sensitive fields:

```
SSN
Bank account
Medical information
Private identifiers
```

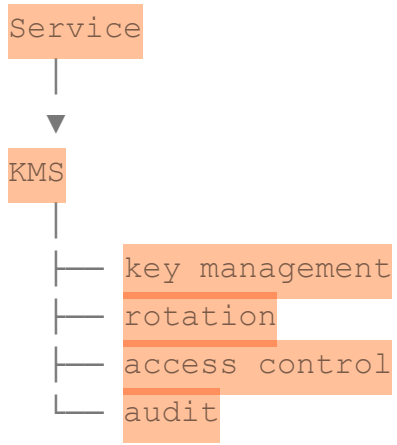
Architecture:



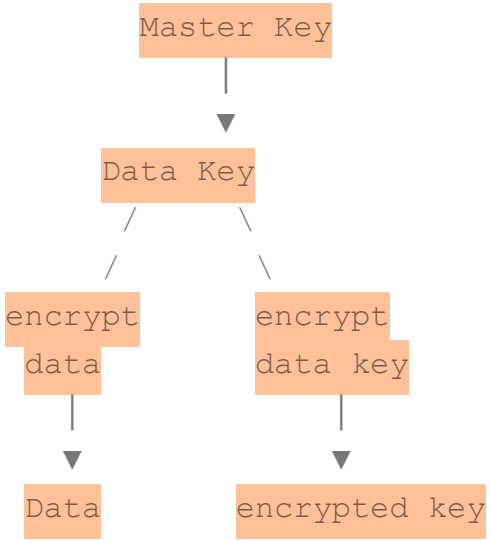
21. KMS architecture

Don't make every service manage raw encryption keys.

Instead:



A common pattern is envelope encryption:



The application stores the encrypted data key alongside encrypted data.

---

22. Database security

Security should exist at multiple levels.

```
Network
   ↓
Authentication
   ↓
Authorization
   ↓
Database roles
   ↓
Row-level security
   ↓
Encryption
   ↓
Audit
```

For multi-tenant systems:

```
tenant_id
```

must be treated as a security boundary.

A dangerous architecture:

```
SELECT *
FROM orders;
```

where the application is responsible for filtering every time.

Better defense in depth:

```
Application authorization
+
database-level tenant isolation
```

For PostgreSQL, Row-Level Security can be useful.

---

23. Multi-tenant security

Imagine:

```
Tenant A
Tenant B
Tenant C
```

Never rely on:

```
WHERE tenant_id = ?
```

alone.

Use multiple controls:

```
Authentication
      ↓
Tenant resolution
      ↓
Authorization
      ↓
Application filtering
      ↓
DB RLS
      ↓
Encryption
```

For high-isolation tenants, you may go further:

```
shared DB
    ↓
separate schema
    ↓
separate database
    ↓
separate cluster
    ↓
separate account/project
```

Isolation level should correspond to the threat model and compliance requirements.

---

24. Message queue security

Security doesn't stop at REST APIs.

For Kafka:

```
Producer
   |
  TLS
   |
Authentication
   |
Authorization
   ▼
Kafka
```

You need:

```
TLS
SASL / workload identity
ACLs
Topic-level authorization
Consumer-group authorization
Encryption
Audit
```

Example:

```
Order Service
     ↓
orders.created
```

Payment Service:

```
READ orders.created
```

But:

```
Payment Service
X WRITE orders.created
```

This is least privilege.

---

25. Object storage security

A common breach pattern is accidental public storage.

Use:

```
Private bucket
      ↓
IAM policy
      ↓
Application
      ↓
Short-lived signed URL
      ↓
Client
```

Instead of:

```
Public bucket
```

For example:

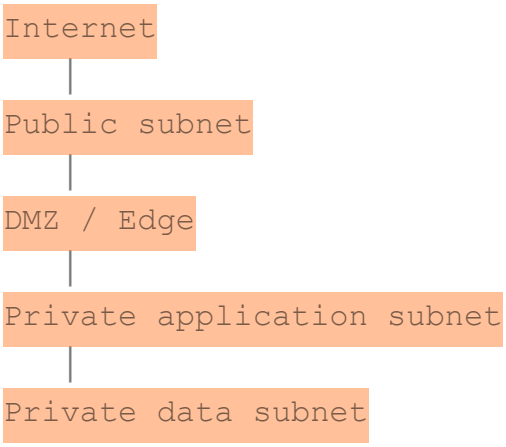
```
Client
  ↓
GET /documents/123
  ↓
Authorization
  ↓
Generate short-lived signed URL
  ↓
Object storage
```

---

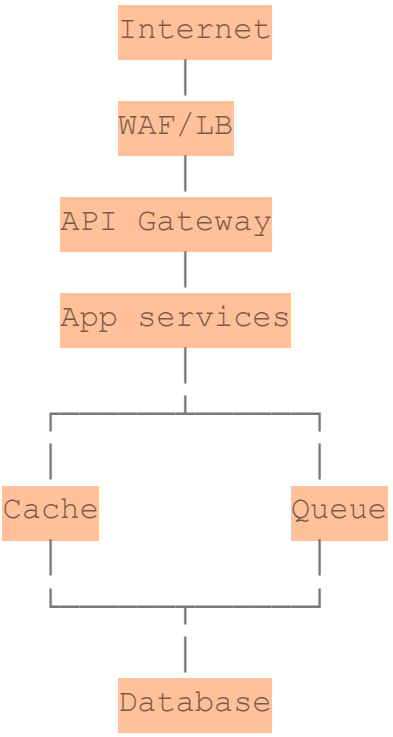
26. Network segmentation

Don't put everything in one network.

Typical architecture:



For example:



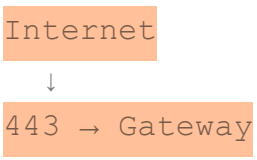
Database should generally not be directly reachable from the Internet.

---

27. Security groups / firewall

Use deny-by-default.

For example:



```
Gateway
  ↓
8443 → Application

Application
  ↓
5432 → PostgreSQL

Application
  ↓
6379 → Redis
```

Not:

```
ANY → ANY
```

Least privilege applies to networks too.

---

28. Egress security

Engineers often focus only on ingress.

But a compromised service can attack the Internet.

Example:

```
Compromised Service
        |
        ├──► attacker.com
        ├──► crypto-mining pool
        ├──► C2 server
        └──► internal systems
```

Use:

```
Egress firewall
DNS filtering
Network policies
Proxy
```

```
Allowlists
Service identity
```

For example:

```
Payment Service
      ↓
Only allowed:
      payment-provider.com
```

---

29. Kubernetes security

For Kubernetes, security exists at several layers:

```
Cloud account
      ↓
Cluster
      ↓
Node
      ↓
Pod
      ↓
Container
      ↓
Application
```

Important controls:

```
RBAC
NetworkPolicy
Pod Security
Secrets management
Image scanning
Admission control
Runtime security
Service accounts
Workload identity
mTLS
Audit logging
```

Avoid giving:

```
ServiceAccount
    cluster-admin
```

unless absolutely necessary.

---

30. Container/image security

CI/CD pipeline:

```
Developer
    ↓
Git
    ↓
CI
    ↓
SAST
    ↓
Dependency scanning
    ↓
Container scanning
    ↓
SBOM
    ↓
Image signing
    ↓
Registry
    ↓
Admission control
    ↓
Kubernetes
```

This is important because security begins **before deployment**.

---

31. Supply-chain security

A modern system can be compromised through:

```
Open-source dependency
Docker image
Build tool
CI plugin
Package registry
Developer dependency
```

Therefore:

```
Dependency scanning
+
SBOM
+
signed artifacts
+
trusted registries
+
reproducible builds
+
CI/CD access controls
```

are part of system security.

---

32. Audit logging

Security decisions need evidence.

Log events such as:

```
Login
Logout
Failed login
MFA failure
Password reset
Permission changes
Role changes
Token issuance
Token revocation
Admin operations
```

```
Sensitive data access
Configuration changes
```

But don't blindly log secrets or sensitive payloads.

Bad:

```
password=...
access_token=...
credit_card=...
```

---

33. Immutable audit trail

For high-value operations:

```
Application
    ↓
Audit Event
    ↓
Kafka
    ↓
Immutable storage
    ↓
SIEM
```

Example:

```
USER_ROLE_CHANGED

actor = admin-123
subject = user-456
old_role = USER
new_role = ADMIN
timestamp = ...
request_id = ...
```

This supports:

```
forensics
compliance
incident investigation
non-repudiation
```

---

34. Observability + security

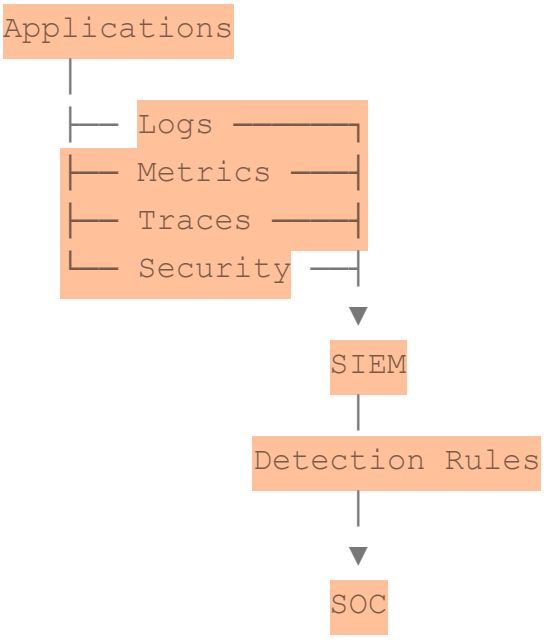
Normal observability:

```
Metrics
Logs
Traces
```

Security observability adds:

```
Security events
Authentication anomalies
Authorization failures
Network anomalies
Privilege escalation
Suspicious API behavior
```

Architecture:



---

35. Detecting compromised accounts

Authentication success doesn't mean the activity is legitimate.

Suppose:

```
User normally:
India → 10 requests/min
```

Suddenly:

```
India
  ↓
Singapore
  ↓
100,000 requests/min
  ↓
massive data download
```

Security systems can detect:

```
Impossible travel
Unusual device
Unusual IP
Abnormal request rate
Abnormal resource access
Privilege escalation
```

Then:

```
Risk score ↑
      ↓
MFA
      ↓
Session revocation
      ↓
Account lock
```

---

36. DDoS protection

Think about DDoS at multiple layers.

Network layer

```
Volumetric attack
```

Protect with:

```
Cloud DDoS protection
Anycast
CDN
Traffic scrubbing
```

Transport layer

```
SYN flood
```

Protect with:

```
SYN cookies
load balancers
connection limits
```

Application layer

```
GET /expensive-report
```

Protect with:

```
WAF
rate limiting
quotas
caching
```

```
request validation
circuit breakers
```

---

37. Protect expensive endpoints

Suppose:

```
POST /generate-report
```

takes 30 seconds of CPU.

An attacker sends:

```
10,000 requests/sec
```

Even if the API is authenticated, you can still be attacked.

Use:

```
Rate limiting
Quota
Async processing
Job queue
Priority queues
Caching
Concurrency limits
```

Architecture:

```
Client
  ↓
API
  ↓
Validate + authorize
   ↓
Create Job
   ↓
Queue
```

```
  ↓
Workers
```

Now the attacker cannot directly consume unlimited CPU.

---

38. Security and asynchronous architecture

Security context must sometimes propagate through queues.

Example:

```
User
 ↓
API
 ↓
Order Service
 ↓
Kafka
 ↓
Payment Worker
```

The worker needs enough context to make the correct authorization/business decision.

But don't blindly forward the entire user JWT.

Instead, publish an explicit security context:

```
{
  "actorId": "user-123",
  "tenantId": "tenant-7",
  "requestId": "req-123",
  "operation": "CREATE_ORDER"
}
```

Then enforce service-specific permissions.

---

39. Security for WebSockets

Since you've been studying WebSocket scaling, this is an important interview extension.

Architecture:

```
Client
  ↓
WAF
  ↓
LB
  ↓
WebSocket Gateway
  ↓
Connection Manager
  ↓
Services
```

At connection establishment:

```
TLS
 ↓
Authentication
 ↓
Authorization
 ↓
Rate limit
 ↓
Connection established
```

Then enforce:

```
Maximum connections/user
Maximum messages/sec
Maximum message size
Idle timeout
Per-channel authorization
```

Don't assume that authenticating the WebSocket connection automatically authorizes every message.

---

40. MQTT security

For an MQTT system:

```
Device
  ↓
TLS
  ↓
MQTT Broker
  ↓
Authentication
  ↓
Topic Authorization
```

Example:

```
device/123/telemetry
```

Device 123 should be allowed:

```
PUBLISH device/123/telemetry
```

but not:

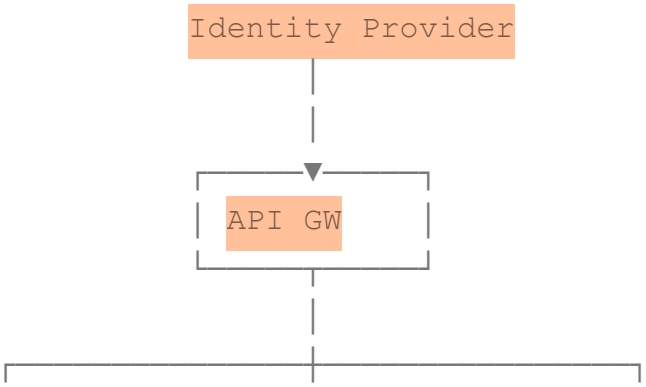
```
PUBLISH device/456/telemetry
```

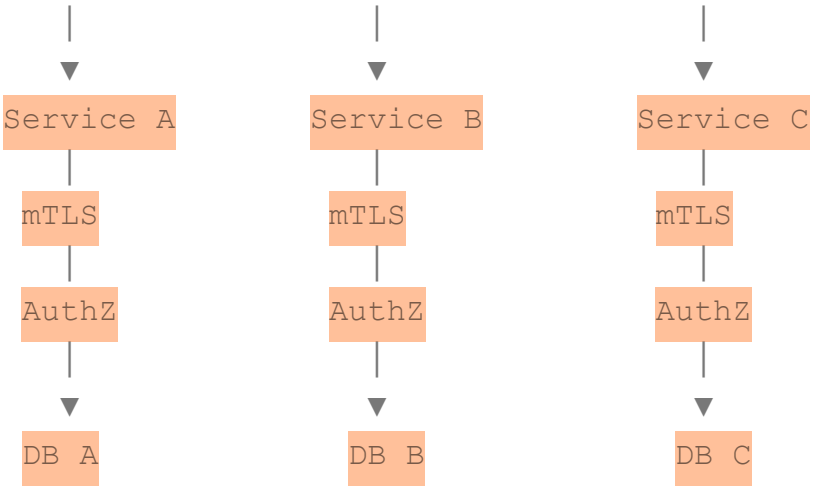
Topic ACLs are therefore a major security control.

---

41. Security for microservices

A strong microservice security architecture looks like:





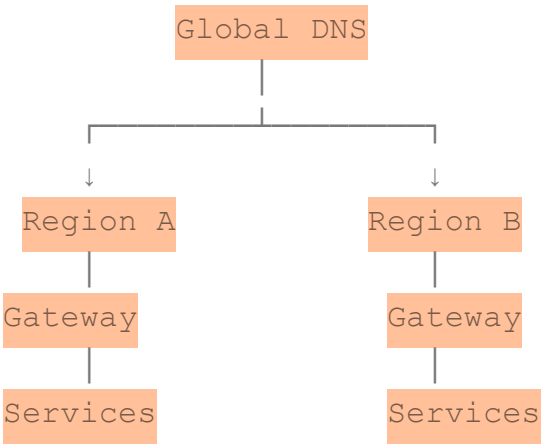
Plus:

```
Secrets Manager
KMS
SIEM
Audit
WAF
DDoS
Rate Limiter
Network Policies
```

---

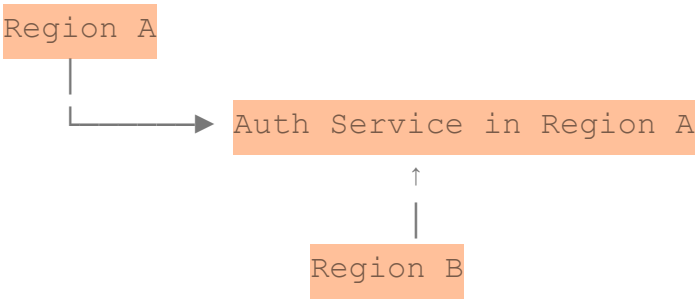
42. Multi-region security

Suppose:



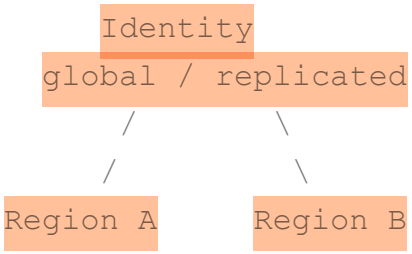
Security infrastructure must also be regional/highly available.

Avoid:



because Region B becomes dependent on Region A.

Instead:



Policies, keys, revocation state, and security telemetry require careful consistency decisions.

---

43. Token revocation in multi-region systems

This is a classic distributed-systems trade-off.

Suppose a user is compromised.

You revoke the user's token in:

```
Region A
```

But:

```
Region B
```

doesn't know immediately.

You have options:

Short-lived tokens

Reduce exposure window.

Central introspection

Every request asks:

```
Is token valid?
```

Strong consistency but adds latency/dependency.

Distributed revocation cache

```
Region A
   ↓
revocation event
   ↓
Kafka
   ↓
Region B
```

Eventually consistent.

Hybrid

```
short-lived access tokens
+
distributed revocation
+
risk-based immediate invalidation
```

The correct choice depends on security requirements.

---

44. Availability vs security

Security controls themselves can become failure points.

Suppose:

```
Application
     ↓
Authorization Service
```

Authorization service fails.

Should the application:

```
FAIL OPEN?
```

or:

```
FAIL CLOSED?
```

For security-sensitive operations:

```
FAIL CLOSED
```

is usually safer.

But this can hurt availability.

For example:

```
READ public profile
```

might tolerate a different strategy than:

```
TRANSFER ₹10,00,000
```

So classify operations by risk.

---

45. Security should be risk-based

Not every operation needs the same controls.

For example:

Operation	Security level
Read public content	Low
Read private profile	Medium
Change password	High
Add bank account	Very high
Transfer money	Very high
Grant admin role	Critical

You can then apply:

```
Low:
authentication

Medium:
authentication + authorization

High:
authentication + authorization + MFA

Critical:
MFA + step-up authentication +
strong authorization + audit +
transaction controls
```

---

46. Step-up authentication

Suppose a user is already logged in.

They attempt:

```
Change email
```

Maybe normal authentication is enough.

But:

```
Transfer money
```

requires:

```
Step-up MFA
```

Flow:

```
Existing session
      ↓
Sensitive operation
      ↓
Risk assessment
      ↓
MFA
      ↓
Short-lived elevated authorization
      ↓
Operation
```

This reduces the impact of stolen sessions.

---

47. API security

Every API should have:

```
Authentication
Authorization
Input validation
Output filtering
Rate limiting
Request size limit
Timeout
Idempotency
Audit
```

For example:

```
POST /payments
```

should potentially require:

```
Valid identity
+
tenant authorization
+
payment permission
+
amount limits
+
idempotency key
+
fraud/risk checks
+
audit event
```

---

48. Idempotency is also a security control

Consider:

```
POST /transfer
```

An attacker or network retry could cause duplicate execution.

Use:

```
Idempotency-Key: abc123
```

Architecture:

```
Request
   ↓
Idempotency Store
   ↓
Already processed?
   ├── Yes → return previous result
```

```
└── No
      ↓
   execute
      ↓
   store result
```

This protects correctness as well as reducing abuse.

---

49. Secure caching

Caches can leak data.

Bad:

```
GET /profile
```

and cache key:

```
/profile
```

Then:

```
User A → cached response
User B → receives User A's data
```

Use security-aware cache keys:

```
tenant:user:resource
```

or avoid caching sensitive responses.

Also consider:

```
Cache poisoning
Authorization bypass through cache
Stale authorization
Sensitive data retention
```

---

50. Security headers

For web applications:

```
Content-Security-Policy
Strict-Transport-Security
X-Content-Type-Options
Referrer-Policy
Permissions-Policy
```

and appropriate cookie settings:

```
Secure
HttpOnly
SameSite
```

These are application-layer controls that complement gateway/WAF security.

---

51. Browser security architecture

For browser applications:

```
Browser
  ↓
TLS
  ↓
CDN/WAF
  ↓
Frontend
  ↓
BFF/API
  ↓
Backend
```

Consider:

```
XSS
CSRF
CORS
Clickjacking
Token theft
Session fixation
Open redirects
```

A Backend-for-Frontend architecture can sometimes reduce browser exposure to long-lived tokens:

```
Browser
   |
   | secure session cookie
   ▼
BFF
   |
   | server-side credentials/tokens
   ▼
Backend APIs
```

---

52. Security at the database layer

Database credentials should not be shared by every service.

Bad:

```
All services
     ↓
same DB user
```

Better:

```
Order Service
   ↓
order_db_user

Payment Service
   ↓
payment_db_user
```

```
Reporting Service
    ↓
read_only_user
```

Then compromise of one service doesn't automatically grant full database access.

---

53. Least privilege everywhere

This is perhaps the single most reusable security principle.

Apply it to:

```
Users
Services
Database accounts
Cloud IAM
Kubernetes service accounts
Kafka consumers
Storage
Networks
Secrets
CI/CD
Developers
```

Example:

```
Order Service
```

should not have:

```
*:*:*
```

permissions.

It should have:

```
orders:read
orders:write
```

and perhaps:

```
customer:read
```

but not:

```
payment:admin
```

---

54. Security boundaries

When designing a system, explicitly identify boundaries:

```
Internet
   |
   | trust boundary
   ▼
Edge
   |
   | trust boundary
   ▼
Application
   |
   | trust boundary
   ▼
Data
```

Also:

```
Tenant A ─── trust boundary ─── Tenant B
```

and:

```
Service A ─── trust boundary ─── Service B
```

This makes the threat model much easier to reason about.

---

55. Scaling security itself

This is where the system-design question becomes interesting.

Security components must scale just like application components.

Identity

```
Multiple IdP instances
+
distributed session/token infrastructure
+
regional replicas
```

API Gateway

```
Horizontal scaling
```

WAF

```
Distributed edge
```

Rate limiting

```
Distributed counters
+
local token buckets
```

Authorization

```
Distributed policy enforcement
+
cached policies
```

Secrets

```
Highly available secret manager
+
local caching where appropriate
```

Audit

```
Kafka
+
partitioning
+
immutable storage
```

SIEM

```
stream processing
+
horizontal consumers
```

---

56. Security event pipeline at scale

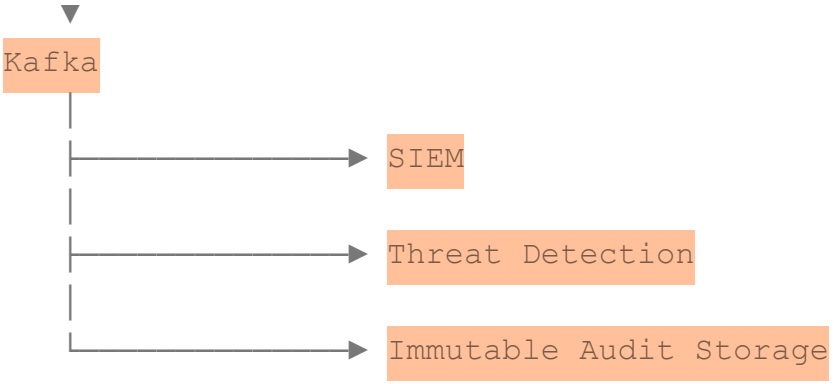
Imagine:

```
10,000 services
1 million requests/sec
```

You cannot synchronously write every security event to a central database.

Instead:

```
Services
   |
   ▼
Local/Event Agent
   |
```



Partition by:

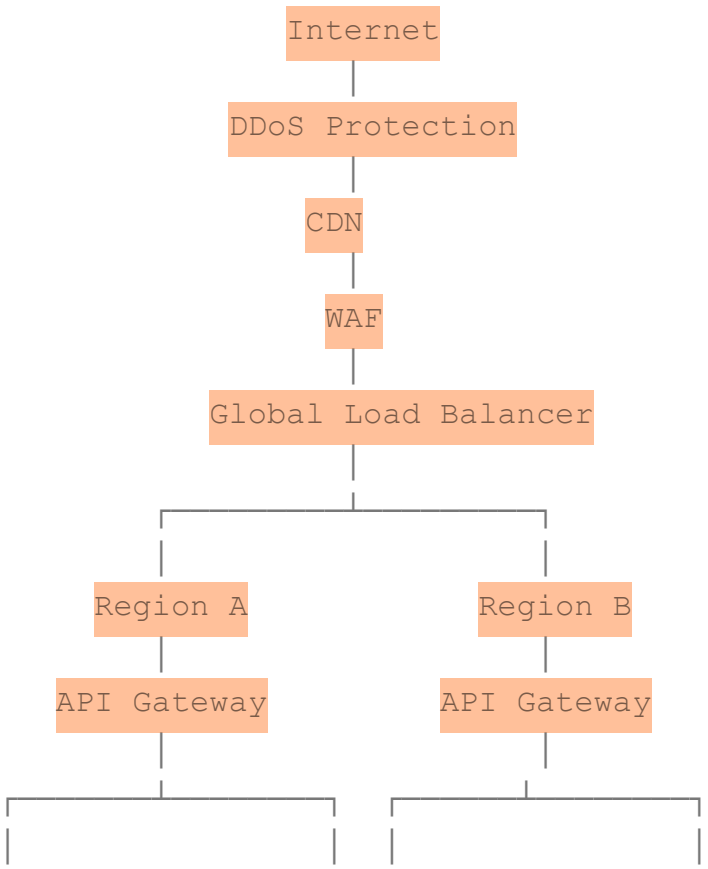
```
tenant
service
region
event type
```

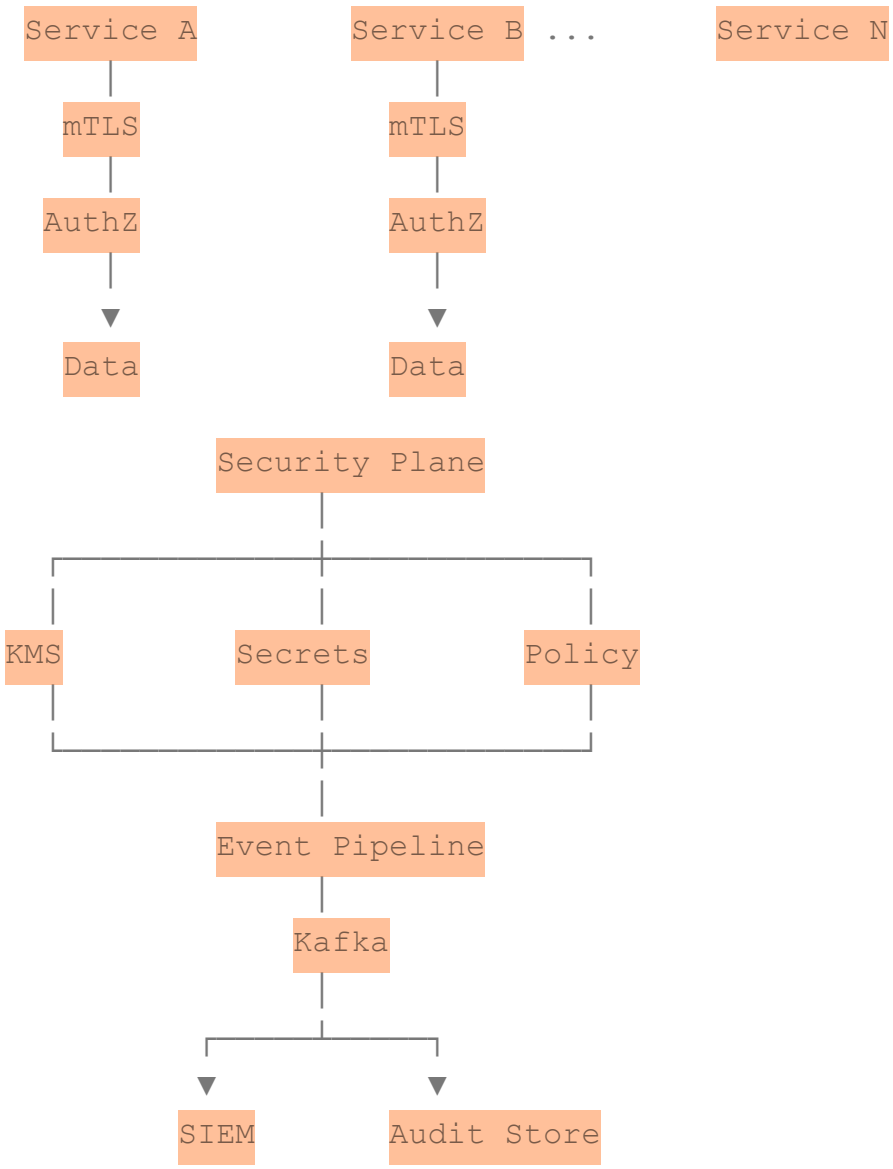
depending on access and ordering requirements.

---

57. Security architecture for 1M requests/sec

A reasonable high-level design:





---

58. Security control plane vs data plane

This is an excellent Staff-level distinction.

Control plane

Defines:

```
Identity
Policies
Certificates
Keys
Configuration
Roles
```

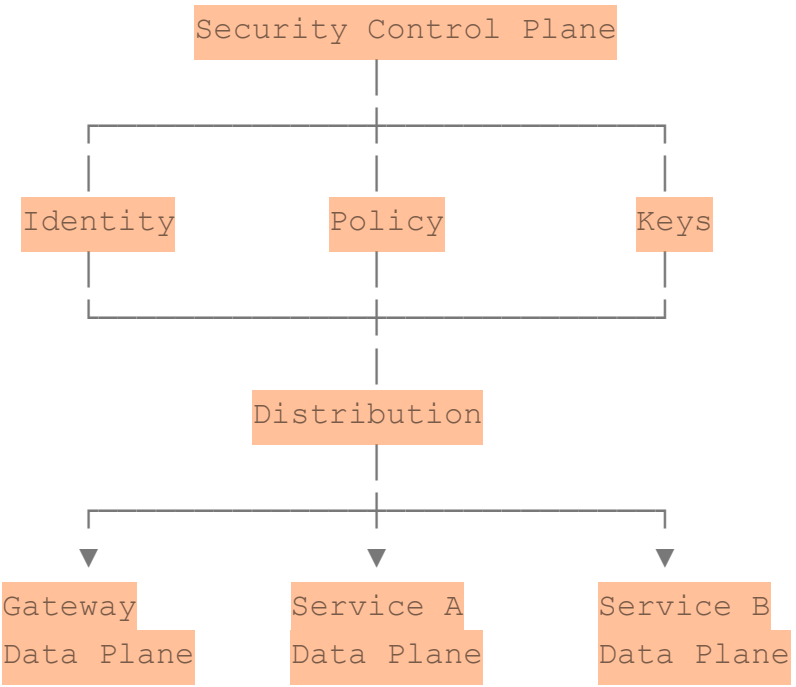
```
Permissions
Security rules
```

Data plane

Enforces:

```
Authentication
Authorization
Network policy
Traffic policy
Encryption
Rate limiting
```

So:

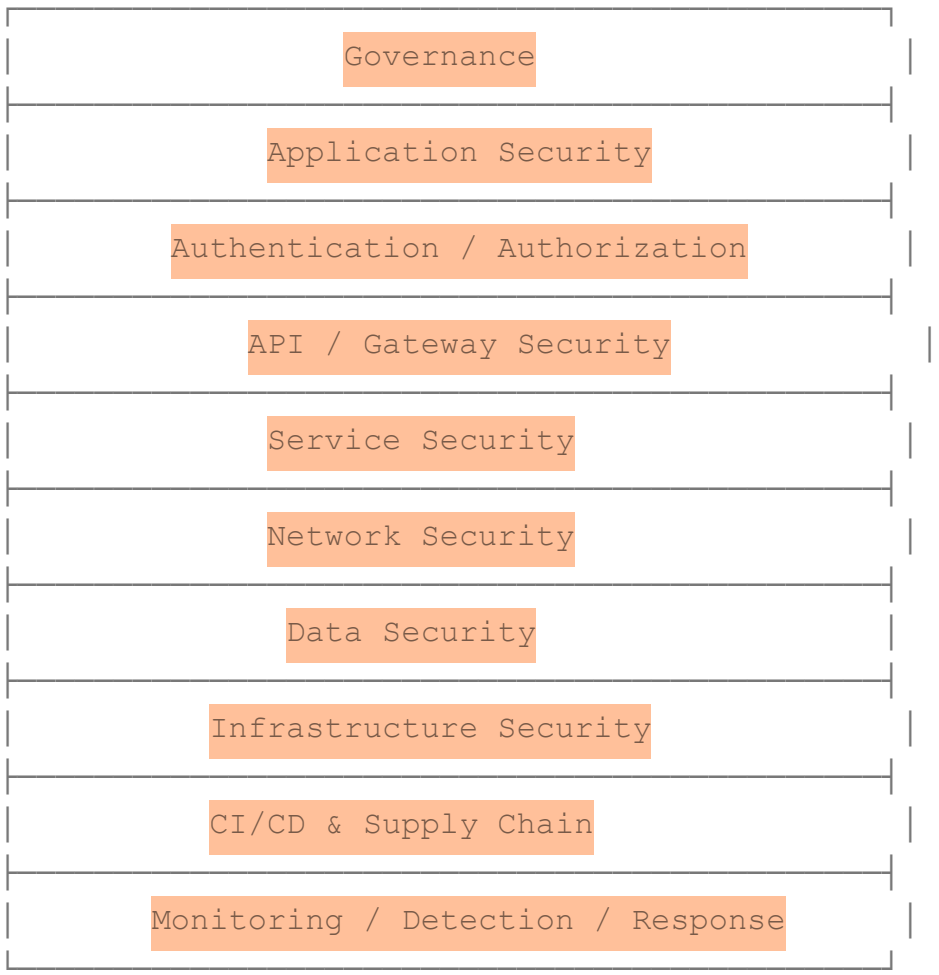


This architecture scales much better than putting all security decisions into one central synchronous service.

---

59. Security architecture layers

For interviews, I recommend remembering this stack:



---

60. Defense in depth

Never depend on one control.

Suppose an attacker steals a user token.

You still want:

```
Token stolen
    ↓
API Gateway
    ↓
Token validation
    ↓
Authorization
    ↓
Tenant isolation
    ↓
Rate limit
```

```
    ↓
Resource-level authorization
    ↓
Database authorization
    ↓
Audit
    ↓
Anomaly detection
```

If one layer fails, another should limit the blast radius.

---

61. Blast-radius reduction

This is another very important Staff-level concept.

Assume:

```
Service A compromised
```

A well-designed system should prevent:

```
Service A
    ↓
all databases
    ↓
all tenants
    ↓
all services
```

Instead:

```
Service A
    ↓
limited IAM permissions
    ↓
limited network access
    ↓
limited database access
    ↓
limited tenant scope
```

```
    ↓
short-lived credentials
```

Therefore:

> **Security isn't only about preventing compromise; it is also about limiting the consequences of compromise.**

---

62. Incident response

Eventually assume something will be compromised.

You need mechanisms to:

```
Detect
  ↓
Contain
  ↓
Investigate
  ↓
Eradicate
  ↓
Recover
  ↓
Learn
```

Technical controls might include:

```
Revoke tokens
Disable accounts
Rotate credentials
Rotate keys
Quarantine workload
Block IP
Block service identity
Isolate network segment
Rollback deployment
Restore data
```

These should be automated wherever safely possible.

---

63. A useful security decision framework

For every component, ask these questions:

1. Who is calling?

```
Authentication
```

2. Are they allowed?

```
Authorization
```

3. What are they allowed to access?

```
Resource-level authorization
```

4. How much can they do?

```
Rate limit / quota
```

5. Can the communication be trusted?

```
TLS / mTLS
```

6. Can the data be exposed?

```
Encryption
```

7. Can the activity be detected?

```
Audit / telemetry
```

8. What happens if this component is compromised?

```
Blast-radius analysis
```

9. What happens if the security system itself fails?

```
Fail-open/fail-closed analysis
```

10. Can we revoke access quickly?

```
Token / credential / certificate revocation
```

---

64. How I would answer this in an interview

If an interviewer asks:

> **"How would you integrate security into this large-scale system?"**

A strong answer can proceed in this order:

Step 1 — Threat model

```
Identify assets, actors, attack surfaces and trust boundaries.
```

Step 2 — Edge security

```
DDoS → CDN → WAF → LB → API Gateway
```

Step 3 — Authentication

```
OAuth/OIDC for users
Workload identity/mTLS for services
```

Step 4 — Authorization

```
RBAC/ABAC/ReBAC
+
resource-level authorization
+
tenant isolation
```

Step 5 — Service security

```
mTLS
least-privilege IAM
network policies
service-level authorization
```

Step 6 — Data security

```
TLS
encryption at rest
KMS
secrets manager
DB authorization
backup encryption
```

Step 7 — Abuse protection

```
rate limiting
quotas
bot protection
request validation
concurrency limits
```

Step 8 — Observability

```
audit logs
security events
```

```
SIEM
anomaly detection
```

Step 9 — Resilience

```
HA
multi-region
distributed policy enforcement
credential rotation
failure-mode analysis
```

Step 10 — Incident response

```
revocation
credential rotation
isolation
forensics
recovery
```

Then explicitly say:

> **"I would centralize security policy, identity, key management, and security telemetry, but distribute enforcement to gateways, services, workloads, and data stores. That prevents the security layer itself from becoming a scalability or availability bottleneck."**

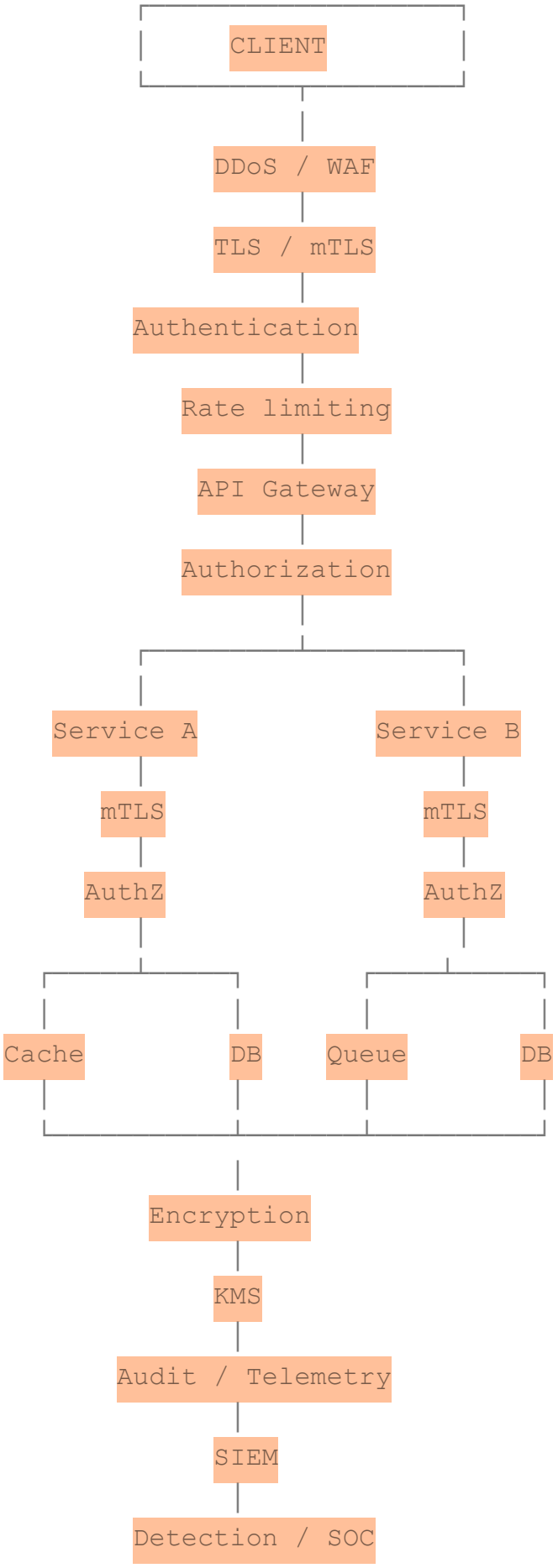
That is a particularly strong architectural principle.

---

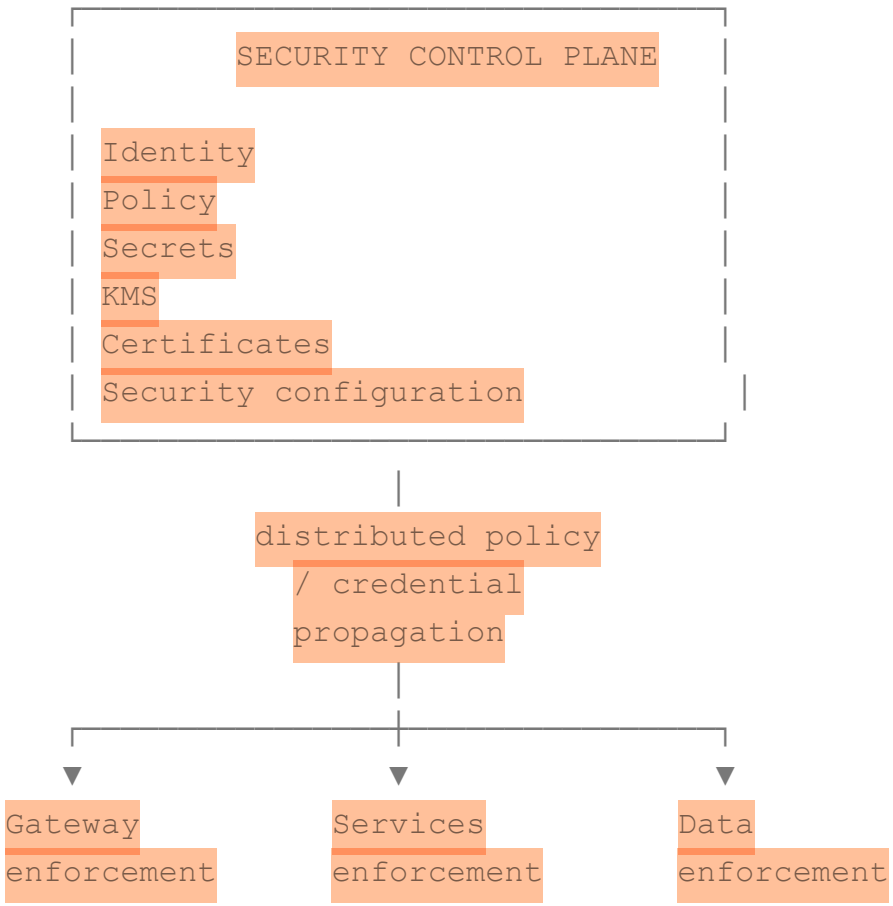
65. The mental model I recommend for your system-design preparation

Given the kinds of distributed-system topics you've been working through—PostgreSQL, RabbitMQ, WebSockets, MQTT, load balancing, multi-region resilience—I'd treat **security as a reusable overlay rather than another isolated system-design topic**.

For almost every design, run this checklist:



And behind all of this:



The key Staff/Architect-level insight is that **security has both a control plane and a data plane**. Centralizing everything creates bottlenecks; decentralizing everything creates inconsistent policy. The scalable middle ground is **centralized identity/policy/key governance + distributed, fail-safe enforcement + asynchronous security telemetry**.